Correction TP 5: Cluster et GPU

TP de Parallélisme

14 octobre 2026

Tâche 2

Voir `main_std_cpu.cpp` et `std_cpu_job.sh`.

Temps d'exécution : environ 165 s.

Bonus

Voir `main_omp_cpu.cpp` et `omp_cpu_job.sh`.

Temps d'exécution : environ 165 s.

Tâche 3

Voir `main_std_gpu.cpp, main_omp_gpu.cpp, std_gpu_job.sh, omp_gpu_job.sh`.

La compilation du code utilisant OpenMP peut être faite avec `clang++` ou `nvc++`.

- OpenMP : environ 1 s.
- stdpar : environ 1 s.

Note : dans le cas des algorithmes standards, un vecteur d'indices est utilisé pour itérer sur les différents pixels. La création et l'initialisation de ce vecteur avec `std::iota()` prennent un temps significatif comparé au temps d'exécution dans ce cas. La mesure de performance commence donc après l'initialisation de ce vecteur intermédiaire.

Tâche 4

Voir `cpu_study_job.sh, run_cpu_study.sh`. Le script `run_cpu_study.sh` soumet plusieurs jobs utilisant le même script `cpu_study_job.sh`, mais en précisant le nombre de CPU par tâche dans la ligne de commande. Il peut directement être exécuté sur le nœud de login sans passer par `sbatch`, car il n'exécute pas le job lui-même.

Résultat :

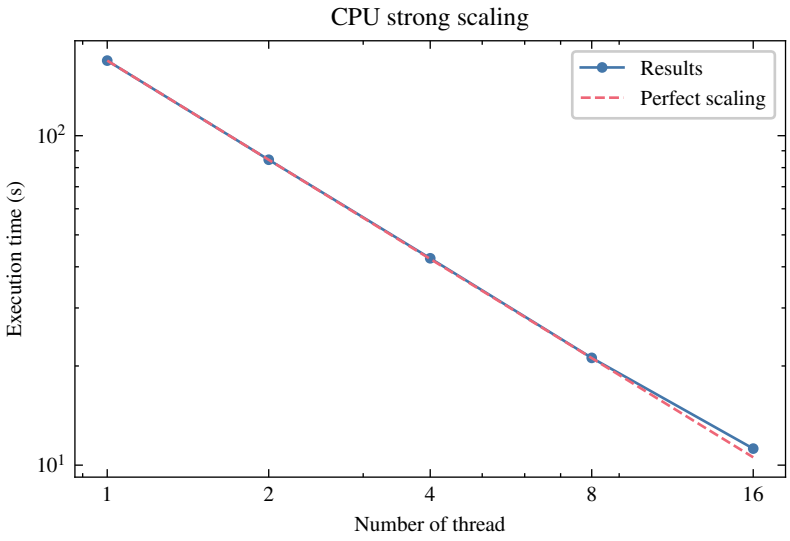


FIGURE 1 – Évolution du temps d'exécution en fonction du nombre de threads utilisés avec une taille de problème constante. Le temps d'exécution avec 1 thread est utilisé comme référence pour le scaling théorique.

Tâche 5

Le script `gpu_study_job.sh` exécute les 3 simulations dans le même job. Il doit donc être soumis avec `sbatch`.

Résultat :

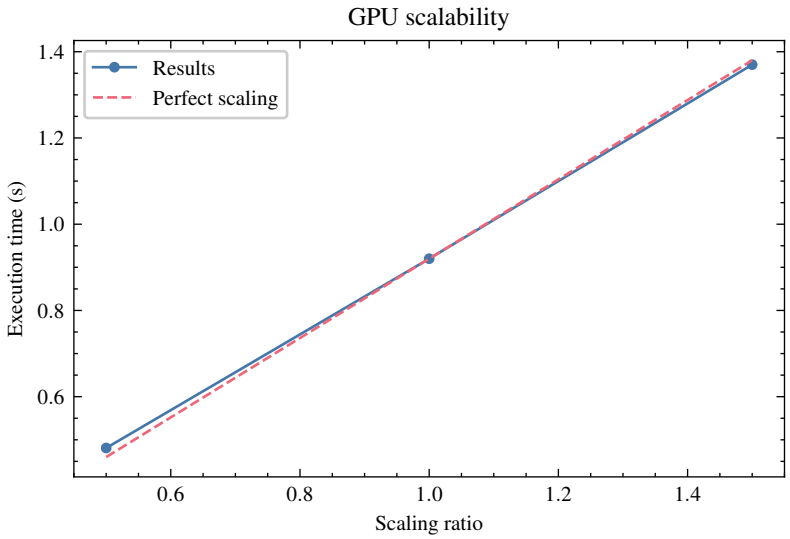


FIGURE 2 – Évolution du temps d'exécution sur GPU en fonction de la taille du problème. Le ratio 1 est utilisé comme référence pour le scaling théorique.